College Event Management System: Build Playbook

Prepared for Binod. Stack: MERN. Paste the prompts into Cursor in Agent mode, in order.

1. Gaps closed (final decisions)

- **Backend:** Node 20 + Express 4 + MongoDB (Mongoose 8). The full API contract lives in docs/SPEC.md, created in Prompt 0.
- **Data models:** User, Event, Registration, with fields and indexes defined in the SPEC.
- **Attendance:** the admin marks it from a per-event checklist, only after the event has started. QR check-in is listed as future scope.
- **Registration rules:** one active registration per student per event (unique index); a full event returns 409; no waitlist; students can cancel until the event starts; re-registering after a cancel reactivates the same record; registration closes at start time.
- **Real-time updates:** no WebSockets. Data is fetched on page load and refetched after every mutation. State this in your report as a deliberate scope decision.
- **Token storage:** JWT (1 day) in localStorage, sent as an Authorization Bearer header, matching the SRS. Trade-off: readable by XSS. Mitigations: React output escaping, no dangerouslySetInnerHTML, short expiry. httpOnly cookies are listed as the production hardening step.
- **Libraries:** Vite, React 18, React Router 6, Tailwind 3, Axios, React Hook Form + Zod, react-hot-toast, Context API for auth. Server: Zod, bcryptjs, jsonwebtoken, helmet, cors, express-rate-limit, Vitest + Supertest + mongodb-memory-server.

2. How to run the chain

1. Create an empty folder, open it in Cursor, and use Agent mode.
2. Paste one prompt at a time. Do not move on until its acceptance checks pass.
3. Commit after each prompt (git commit -m 'prompt N').
4. If something breaks, paste the error back with: Fix the root cause, not the symptom, then re-run the acceptance checks.

3. The prompts

Prompt 0: Foundation, spec, and rules

```
ROLE
You are a senior full-stack engineer setting up a monorepo for a graded
college project. Correctness, clarity and demo-readiness matter more than
cleverness.

TASK
Create the skeleton and two files every later task depends on. Write NO
application code yet.
1. Create client/ and server/ (each with a .gitkeep), docs/, a root
.gitignore (node_modules, .env, dist, coverage) and a root README.md (title
plus one paragraph).
2. Create .cursor/rules/project.mdc with alwaysApply: true and these rules:
read docs/SPEC.md before every task; JavaScript ES modules only, no
TypeScript; never invent endpoints, fields or routes that are not in the
SPEC; small focused files; every async handler wrapped in asyncHandler; no
secrets in code; no console.log except server startup; after each task run
the relevant lint, test or build command and report the real output; if the
SPEC is ambiguous, stop and ask instead of guessing.
3. Create docs/SPEC.md with EXACTLY the content between the markers below.

=== BEGIN SPEC ===
# SPEC: College Event Management System (source of truth)

## Stack
Client: Vite, React 18, React Router 6, Tailwind CSS 3, Axios, React Hook
Form + Zod, react-hot-toast. Auth state via Context API.
Server: Node 20, Express 4, Mongoose 8, Zod, bcryptjs, jsonwebtoken, helmet,
cors, express-rate-limit. Tests: Vitest, Supertest, mongodb-memory-server.
Server env: PORT=5000, MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN=1d,
CLIENT_ORIGIN=http://localhost:5173, ADMIN_EMAIL, ADMIN_PASSWORD.
Client env: VITE_API_URL=http://localhost:5000/api

## Roles
Guest (no token): list and view published events only. Student: register,
cancel, personal dashboard. Admin: event CRUD, registrations, attendance,
stats. Public signup always creates a student. The admin exists only via the
seed script.
```

```
## Models
User: name (2-80), email (unique, lowercase), passwordHash, role
(student|admin, default student), department (optional), rollNumber
(optional), timestamps.
Event: title (3-120), description (10-2000), category
(Technical|Cultural|Sports|Workshop|Seminar|Other), venue, startsAt, endsAt,
capacity (int 1-10000), registeredCount (int, default 0), status
(draft|published|cancelled), createdBy (User), timestamps. Serialized field
phase = upcoming|ongoing|completed, computed from dates. Index on startsAt
and status.
Registration: event (ref), student (ref), status (registered|cancelled),
attended (bool, default false), attendedAt, registeredAt, cancelledAt. Unique
compound index (event, student).

## Business rules
R1 Guests and students see only published events. Admin sees all.
R2 Register only if: event published, now < startsAt, registeredCount <
capacity. Duplicate active registration: 409 'Already registered'. Full: 409
'Event is full'. Closed: 400 'Registration closed'.
R3 Capacity is enforced atomically: findOneAndUpdate on the event with
registeredCount < capacity and $inc 1. If creating or reactivating the
registration then fails, roll the counter back. No transactions.
R4 Cancel only while now < startsAt: status cancelled, cancelledAt set,
counter decremented. Registering again reactivates the same document and
resets attended to false.
R5 Attendance is set by admin only when now >= startsAt and the registration
status is registered.
R6 Event update: capacity may not drop below registeredCount.
R7 Event delete only if it has zero registrations of any status; otherwise
409 with advice to set status cancelled.
R8 No WebSockets. Clients refetch on mount and after mutations.

## API (base /api)
Success: {success:true, data, message?}. Error: {success:false, message,
errors?}. Codes: 400 validation, 401 unauthenticated, 403 wrong role, 404,
409 conflict, 500. Header: Authorization: Bearer <token>. JWT payload {sub,
role}.
Auth: POST /auth/register {name,email,password (min 8,
letter+number),department?,rollNumber?} -> {token,user}. POST /auth/login
{email,password} -> {token,user}. GET /auth/me (auth) -> user.
Events: GET /events?search&category&from&to&page=1&limit=9 (public; published
only; from defaults to now and matches endsAt >= from; to matches startsAt <=
to; sorted by startsAt ascending) -> {items,page,limit,total,totalPages}. GET
```

```
/events/:id (optional auth) -> event plus myRegistration (null or
{_id,status}) when a token is present.
Admin events: GET /admin/events?search&status&page&limit (admin, all
statuses, newest first). POST /events, PUT /events/:id, DELETE /events/:id
(admin).
Registrations: POST /events/:id/register (student). DELETE
/events/:id/register (student, cancel). GET /registrations/me (student;
populated event; newest first). GET /events/:id/registrations (admin;
populated student {name,email,rollNumber,department}). PATCH
/events/:id/registrations/:registrationId/attendance (admin)
{attended:boolean}.
Stats: GET /admin/stats (admin) -> {totalEvents, publishedEvents,
upcomingEvents, totalRegistrations, attendanceRate (0-100 integer), upcoming:
next 5 published events with registeredCount and capacity}.
Health: GET /health.

## Client routes
/ redirects to /events. /events, /events/:id, /login, /register, /dashboard
(student), /admin, /admin/events, /admin/events/new, /admin/events/:id/edit,
/admin/events/:id/registrations, * is a 404 page.
Post-login: go to the originally requested route if present, else student to
/dashboard, admin to /admin. Unauthenticated on a protected route: /login
with return path. Wrong role: redirect to own home with a toast.
UX: every data view has loading skeleton, empty state and error state with
retry. Forms show field-level errors. Destructive actions use a confirmation
modal.
=== END SPEC ===

ACCEPTANCE
- All files exist and docs/SPEC.md matches the markers' content verbatim.
- Run git init and commit 'chore: foundation'.
- Report the final file tree. Do nothing else.
```

Prompt 1: Backend core

```
ROLE
Senior Node.js engineer. First read docs/SPEC.md fully.

TASK
Build the server foundation in server/. No business features yet.
- npm init with type: module. Install runtime deps (express, mongoose, zod,
bcryptjs, jsonwebtoken, helmet, cors, express-rate-limit, dotenv) and dev
deps (vitest, supertest, mongodb-memory-server). Scripts: dev (node --watch
src/server.js), start, test (vitest run), seed.
- src/config/env.js: validate process.env with Zod, crash with a readable
message if invalid. src/config/db.js: connect helper.
- src/app.js: exports the Express app without listening. Use helmet, cors
restricted to CLIENT_ORIGIN, express.json({limit:'100kb'}), a rate limiter
(100 requests per 15 min) on /api/auth, mount /api routes, notFound handler,
errorHandler. src/server.js connects to DB then listens.
- src/utils: asyncHandler.js, ApiError.js (statusCode, message, errors),
respond.js (ok, created helpers producing the SPEC envelope).
- src/middleware: validate.js (validate(schema, source) parses body or query
with Zod and returns 400 with field-level errors), errorHandler.js mapping:
ApiError, Zod, Mongoose CastError (400), duplicate key 11000 (409),
JsonWebTokenError and TokenExpiredError (401), malformed JSON (400), anything
else (500, no stack in production).
- src/routes/index.js with GET /api/health.
- server/.env.example with every variable from the SPEC.

CONSTRAINTS
Follow the SPEC envelope exactly. Keep files under about 80 lines.

ACCEPTANCE
- npm run dev boots. GET /api/health returns the envelope. An unknown route
returns a 404 envelope. Malformed JSON returns a 400 envelope.
- Show the actual curl output for all three.
```

Prompt 2: User model and authentication

```
ROLE
Senior backend engineer focused on secure auth. Read docs/SPEC.md first.

TASK
Implement authentication in server/.
- src/models/User.js per SPEC (lowercase trimmed email, passwordHash never
returned by toJSON).
- src/modules/auth/{auth.schemas.js, auth.service.js, auth.controller.js,
auth.routes.js}. Register: role is always student; any role in the body is
ignored. Hash with bcryptjs cost 12. Login: return the same message 'Invalid
email or password' for unknown email and wrong password, and compare against
a dummy hash when the user does not exist. signToken uses JWT_SECRET,
JWT_EXPIRES_IN, payload {sub, role}.
- src/middleware/auth.js: authenticate (required; 401 if missing, invalid or
user no longer exists), optionalAuthenticate (sets req.user if a valid token
is present, never fails), authorize(...roles) (403).
- src/scripts/seed.js: for now create the admin from ADMIN_EMAIL and
ADMIN_PASSWORD if absent (idempotent).
- Tests in server/tests/auth.test.js with Vitest, Supertest and mongodb-
memory-server: register success; duplicate email 409; weak password 400; role
escalation attempt ignored; login success and failure; /auth/me with and
without token; expired token 401.

ACCEPTANCE
- npm test is green; paste the output.
- passwordHash never appears in any response.
```

Prompt 3: Events API

```
ROLE
Senior backend engineer. Read docs/SPEC.md first, especially R1, R6, R7 and
the Events endpoints.

TASK
Implement events in server/src/modules/events (schemas, service, controller,
routes) and src/models/Event.js.
- Public GET /events: published only; search is a case-insensitive match on
title with user input regex-escaped; filters category, from (default now,
endsAt >= from), to (startsAt <= to); pagination defaults page 1 and limit 9,
max limit 50; response shape per SPEC.
- GET /events/:id with optionalAuthenticate. Unpublished events return 404
for non-admins. When a token is present include myRegistration (query
Registration if the model exists; otherwise return null and leave a TODO the
next prompt will resolve).
- Admin: GET /admin/events (all statuses, search, status filter, pagination),
POST /events, PUT /events/:id, DELETE /events/:id. Zod rules: title,
description, category, venue required; endsAt after startsAt; startsAt must
be in the future on create; capacity integer 1-10000; status defaults to
draft. Enforce R6 (capacity not below registeredCount) and R7 (delete only
with zero registrations; otherwise 409). Add the phase field to every
serialized event.
- Tests: guests never see drafts or cancelled; admin does; invalid dates
rejected; capacity floor; delete blocked once a registration exists (create
the Registration via the model directly if needed); pagination and filters.

ACCEPTANCE
- npm test green with real output pasted. List every route you added with its
method and role.
```

Prompt 4: Registrations, attendance, stats, rich seed

```
ROLE
Senior backend engineer who cares about concurrency correctness. Read
docs/SPEC.md first, especially R2 to R5.

TASK
Implement src/models/Registration.js (unique compound index on event+student)
and src/modules/registrations.
- POST /events/:id/register (student): enforce R2 and R3. Increment the
counter with a single findOneAndUpdate filtered by status published, startsAt
> now and registeredCount < capacity. Then create the registration, or
reactivate a cancelled one (reset attended and attendedAt). If that second
step fails, decrement the counter. Distinguish the 404, 400 closed, 409 full
and 409 already registered cases with the exact messages from the SPEC.
- DELETE /events/:id/register (student): R4.
- GET /registrations/me (student): populate the event, newest first.
- GET /events/:id/registrations (admin): populate the student with name,
email, rollNumber, department.
- PATCH /events/:id/registrations/:registrationId/attendance (admin): R5.
- GET /admin/stats per SPEC. Wire myRegistration into GET /events/:id.
- Expand src/scripts/seed.js (idempotent, with a --reset flag): 1 admin, 6
students (password Student@123), 10 events across all categories and phases
(past, ongoing, upcoming, one draft, one cancelled, one full), and realistic
registrations including attended ones. Print the seeded credentials at the
end.
- Tests: duplicate registration; full event; a race test with capacity 1 and
two students calling register via Promise.all where exactly one succeeds and
registeredCount equals 1; cancel then re-register; registration closed after
start (create the past event directly via the model to bypass validation);
attendance before start rejected; non-admin attendance 403; stats numbers
correct.

ACCEPTANCE
- npm test green with real output pasted. npm run seed -- --reset runs and
prints credentials.
```

Prompt 5: Client foundation

```
ROLE
Senior React engineer with strong design-system and accessibility habits.
Read docs/SPEC.md first, especially Client routes.

TASK
Build the client foundation in client/ and nothing page-specific yet.
- Scaffold with Vite (React, JavaScript). Install react-router-dom@6, axios,
react-hook-form, zod, @hookform/resolvers, react-hot-toast, tailwindcss@3
with postcss and autoprefixer. Configure Tailwind (indigo primary, slate
neutrals, visible focus ring styles). Add client/.env.example.
- Folders: src/{api,components/{ui,layout},context,hooks,pages,routes,utils}.
- src/api/client.js: axios instance with baseURL from VITE_API_URL; request
interceptor attaches Bearer token from a safe storage util (try/catch around
localStorage); response interceptor on 401 clears the token and redirects to
/login with the return path, except for the login and register calls. Modules
authApi, eventsApi, registrationsApi, adminApi covering every SPEC endpoint,
each returning response.data.data.
- AuthContext: user, token, isBootstrapping, login, register, logout. On
load, if a token exists call /auth/me; show a full-page spinner until
resolved to prevent a flash of the wrong UI.
- routes: ProtectedRoute (redirect with return path) and RoleRoute (wrong
role redirects to own home with a toast). Define ALL SPEC routes with simple
placeholder pages and a real 404 page.
- Layout: Navbar (role-aware links, accessible mobile menu with aria-
expanded), Footer, AppShell with a skip-to-content link, and AdminLayout with
a sidebar.
- UI kit in components/ui: Button (variants, loading state), Input, Select,
Textarea (label, error text wired with aria-describedby), Card, Badge, Modal
(focus trap, Escape closes, aria-modal, returns focus), ConfirmDialog,
Skeleton, EmptyState, ErrorState (retry), Pagination, ProgressBar (role
progressbar with aria values).
- hooks/useFetch.js: returns data, loading, error, refetch; aborts on
unmount. utils/format.js using Intl with en-IN for dates and times.

ACCEPTANCE
- npm run build passes. A guest opening /dashboard lands on /login and the
return path is preserved. Report the file tree and the build output.
```

Prompt 6: Authentication pages

```
ROLE
Senior React engineer. Read docs/SPEC.md and reuse the existing UI kit and
AuthContext.

TASK
Implement /login and /register.
- React Hook Form with Zod resolvers mirroring the server rules (name 2-80,
valid email, password min 8 with a letter and a number, optional department
and rollNumber).
- Show or hide password toggle (a real button with an accessible name).
Correct autocomplete attributes. The submit button is disabled and shows a
spinner while loading.
- Map server 400 field errors onto the fields; show other errors with a toast
and an inline alert.
- After success follow the SPEC post-login redirect rules. Logged-in users
visiting /login or /register are redirected home.
- Layout is a centered card that works from 360px wide.

ACCEPTANCE
- Using the seeded users, log in as student, as admin, with wrong
credentials, and register a new student. Describe the observed result of
each. npm run build passes.
```

Prompt 7: Event discovery and detail

```
ROLE
Senior React engineer with product sense. Read docs/SPEC.md first, especially
R1 to R4.

TASK
Implement /events and /events/:id.
- Listing: responsive grid (1, 2, 3 columns) of EventCard (title, category
badge, date and time, venue, phase badge, capacity ProgressBar with text such
as 42 of 60 registered). Filter bar with a search box debounced 400ms,
category select, and from and to date inputs. Sync all filters and the page
number to URL query parameters so links are shareable. Use Pagination.
Provide loading skeleton, empty state ('No events match your filters' with a
clear-filters button) and error state.
- Detail: title, description, category, date range, venue, capacity, phase,
and a cancelled-event banner.
- A single action area driven by an explicit state machine, with the logic in
one pure function (and unit-testable): guest shows 'Log in to register' (link
to /login with return path); student not registered and open shows Register;
full shows a disabled 'Event full'; registered before start shows 'Cancel
registration' (ConfirmDialog); started or completed shows a disabled
explanatory label; admins see a link to manage the event instead.
- After register or cancel, refetch and toast. Handle 409 and 400 server
messages gracefully by showing them and refetching.

ACCEPTANCE
- Walk through as guest, student and admin and describe what each sees.
Demonstrate registering for a full event and cancelling. npm run build
passes.
```

Prompt 8: Student dashboard

```
ROLE
Senior React engineer. Read docs/SPEC.md. Use only GET /registrations/me.

TASK
Implement /dashboard.
- Summary cards: upcoming registrations, events attended, total registrations (cancelled excluded).
- Tabs (accessible tablist): Upcoming, Past, Cancelled. Each row shows the event, date, venue and a status chip: Registered, Attended, Absent (event has ended and attended is false), Cancelled.
- Cancel action on upcoming rows with ConfirmDialog; refetch afterwards.
- Link each row to its event page. Proper loading, empty state per tab (with a Browse events call to action) and error state.
- Derive all groupings client-side in a pure, unit-tested helper.

ACCEPTANCE
- Log in as a seeded student with mixed registrations and verify each tab and chip. npm run build and the helper's tests pass.
```

Prompt 9: Admin interface

```
ROLE
Senior React engineer building an admin console. Read docs/SPEC.md first,
especially R5 to R7.

TASK
Implement all /admin routes inside AdminLayout.
1. /admin dashboard: stat cards from /admin/stats (total, published, upcoming
events, total registrations, attendance rate) and a list of the next 5 events
with fill-rate bars.
2. /admin/events: table with search, status filter, pagination; status badge;
row actions View registrations, Edit, Delete (ConfirmDialog; show the 409
message from the server verbatim), and a quick Publish or Unpublish toggle.
3. /admin/events/new and /admin/events/:id/edit: one shared EventForm with
React Hook Form and Zod mirroring the server rules. Convert datetime-local
values to ISO correctly and back, with no off-by-timezone bugs. On edit,
capacity minimum equals the current registeredCount and the error message
says so.
4. /admin/events/:id/registrations: header with counts (registered,
attended), client-side search by name or email, table of students with an
attendance checkbox. Checkboxes are disabled before the event starts, with a
visible explanation. The toggle updates optimistically and rolls back with a
toast on failure. Add an Export CSV button that builds the file client-side
from the loaded rows.

ACCEPTANCE
- As the seeded admin: create an event, publish it, register a student for it
in another session, mark attendance on a past event, delete a registration-
free event, and try deleting one with registrations. Report the observed
result of each. npm run build passes.
```

Prompt 10: Hardening, accessibility, and the permission matrix

```
ROLE
Staff engineer doing a pre-release audit for a graded project. Read
docs/SPEC.md.

TASK
Audit and fix, then report.
1. Security tests (server): add tests/permissions.test.js covering EVERY
endpoint as guest, student and admin against the SPEC, and write the table to
docs/TEST_MATRIX.md. Confirm no endpoint leaks passwordHash and that drafts
are never exposed to non-admins.
2. Accessibility (client): keyboard-only walk of every page; labels for every
control; focus moves to the page heading on route change; toasts announced
via aria-live; color contrast at least 4.5:1; focus indicators visible;
modals trap and restore focus. Fix every issue found.
3. Responsive: verify at 360px, 768px and 1280px; no horizontal scroll;
tables scroll inside their container.
4. Robustness: an app-level ErrorBoundary, consistent error and empty states,
no unhandled promise rejections, no unused code or console statements.
5. Run Lighthouse accessibility on /events and /login and fix until at least
90.

ACCEPTANCE
- Output: list of issues found, fixes made, remaining known limitations,
final npm test and npm run build output for both packages.
```

Prompt 11: Documentation, deployment, demo script

```
ROLE
Technical writer and release engineer. Document only what actually exists in
the code; verify against the repository.

TASK
1. README.md: overview, tech stack, prerequisites, setup steps for server and
client, environment variables, scripts, seeded credentials, and how to run
the tests.
2. docs/API.md: generated from the real route files, with method, path, role,
request body, success response, and error cases.
3. docs/TRACEABILITY.md: a table mapping each section of the React frontend
document (sections 4 to 21: SPA architecture, component design, routing,
role-based UI, auth flow, event listing, event detail, student dashboard,
admin dashboard, event management, notifications, responsive design,
accessibility, API handling, security, scalability, deployment) to the
concrete files and features that implement it.
4. docs/LIMITATIONS.md: honest list of scope decisions and future work (no
WebSockets, localStorage token trade-off and httpOnly cookie upgrade, no
waitlist, no QR attendance, no email notifications, calendar view).
5. Deployment: add docs/DEPLOY.md for MongoDB Atlas, the server on Render,
the client on Vercel or Netlify (VITE_API_URL, CLIENT_ORIGIN, SPA fallback
rewrite to index.html), plus a production checklist (strong JWT_SECRET, seed
the admin once, HTTPS).
6. docs/DEMO.md: a 5-minute demo script covering guest browsing, student
registration and cancellation, admin event creation, attendance marking, and
the stats dashboard, with the exact accounts to use.

ACCEPTANCE
- Every command in the README works when followed from a clean clone (state
which ones you verified). No claim in any document that the code does not
support.
```

4. Notes for your viva and report

- Be ready to explain R3: capacity is enforced with one atomic update, so two simultaneous registrations cannot oversell a seat.
- Be ready to explain the localStorage token trade-off and the httpOnly cookie upgrade.
- Use docs/TRACEABILITY.md in your report to show every SRS section is implemented.